ScaleNet: Detailed Technical Architecture
Learnable Multi-Scale Voxelization with MLP and Gumbel-Softmax

1. INPUT POINT CLOUD

Point Cloud: $P = \{p_i\}_{i=1}^{N}$

Each point: $p_i = (x_i, y_i, z_i, r_i)$

where (x, y, z) = position

r = reflectance intensity

2. LEARNABLE VOXEL SCALES (PhD Contribution)

$theta_scale = [\theta_1, \theta_2, \theta_3]$

Initialization:

$\theta_1 = 0.05m$ (fine - small objects)

$\theta_2 = 0.10m$ (medium - cars)

$\theta_3 = 0.20m$ (coarse - large vehicles)

Feed to ScaleNet

PyTorch Implementation:

```
self.voxel_scales = nn.Parameter(
torch.tensor([0.05, 0.
```

3. SCALE ASSIGNMENT NETWORK (MLP)

Input Layer

$p_i = (x, y, z, r)$

Shape:

`[N, 4]`

N = num points

4 = features

(x, y, z, r)

Hidden Layer 1

Linear: 4→64

$h_1 = ReLU(W_1 p_i + b_1)$

Shape:

`[N, 64]`

Parameters:

$W_1 \in \mathbb{R}^{64 \times 4}$

$b_1 \in \mathbb{R}^{64}$

Hidden Layer 2

Linear: 64→32

$h_2 = ReLU(W_2 h_1 + b_2)$

Shape:

`[N, 32]`

Parameters:

$W_2 \in \mathbb{R}^{32 \times 64}$

$b_2 \in \mathbb{R}^{32}$

Output Layer

Linear: 32→3

$z_i = W_3 h_2 + b_3$

Shape:

`[N, 3]`

Logits (unnormalized)

$z_i = [z_i^{(1)}, z_i^{(2)}, z_i^{(3)}]$

One logit per scale

Logits per Point

Example for point i:

$z_i^{(1)} = 2.1$ (fine)

$z_i^{(2)} = 0.5$ (medium)

$z_i^{(3)} = -1.2$ (coarse)

Higher logit =

stronger preference

for that scale

WHY THIS WORKS:

1. MLP learns point-specific scale preferences

2. Gumbel-Softmax makes discrete selection differentiable

3. Learnable θ adapts to dataset/task

4. End-to-end training: gradients flow from detection loss back to voxel scales

5. No manual tuning!

GRADIENT FLOW:

Detection Loss
↓
Feature Aggregation
↓
Gumbel Weights σ
↓
MLP Parameters
↓
Voxel Scales θ

All parameters updated via backpropagation!

4. GUMBEL-SOFTMAX: Differentiable Discrete Selection

Step 1: Sample Noise

Sample: $g_i^{(s)} \sim Gumbel(0, 1)$

$g = -\log(-\log(u))$

where $u \sim Uniform(0, 1)$

Adds stochasticity

(only during training)

Shape: `[N, 3]`

Step 2: Add to Logits

$\tilde{z}_i^{(s)} = z_i^{(s)} + g_i^{(s)}$

Example:

$\tilde{z}_i = [2.3, 0.8, -0.9]$

Perturbed logits

for exploration

Step 3: Softmax + Temp

$\sigma_i^{(s)} = \frac{\exp(\tilde{z}_i^{(s)}/\tau)}{\sum_{s=1}^{3} \exp(\tilde{z}_i^{(s)}/\tau)}$

Temperature $\tau = 1.0$

Lower τ = sharper

Output: probability distribution

$\sum_{s=1}^{3} \sigma_i^{(s)} = 1$

Step 4: Scale Weights

For each point i:

$\sigma_i = [\sigma_i^{(1)}, \sigma_i^{(2)}, \sigma_i^{(3)}]$

Example:

$\sigma_i = [0.73, 0.22, 0.05]$

Point *i* prefers fine scale

(73% weight on θ_1)

5. USING LEARNABLE SCALES IN VOXELIZATION

KEY: Gumbel-Softmax is DIFFERENTIABLE! Gradients flow back through softmax to MLP and θ_{scale}

Scale 1: θ_1

Voxel size: $\approx 0.04m$

Weight: $\sigma_i^{(1)} = 0.73$

⇓ *Voxelize points*

Scale 2: θ_2

Voxel size: $\approx 0.12m$

Weight: $\sigma_i^{(2)} = 0.22$

Final feature: $f_i = \sigma_i^{(1)} \cdot f_i^{(1)} + \sigma_i^{(2)} \cdot f_i^{(2)} + \sigma_i^{(3)} \cdot f_i^{(3)}$

Scale 3: θ_3

Voxel size: $\approx 0.25m$

Weight: $\sigma_i^{(3)} = 0.05$

⇓ *Voxelize points*

where $f_i^{(s)}$ = feature extracted from scale s voxelization